

Week 7 Lab Lab Instructions

Task 1: Encrypted Messaging App Prototype

Requirements:

- User A generates an RSA key pair and shares the public key.
- User B encrypts a secret message using:
 - ✓ AES-256 (with randomly generated key)
 - ✓ Encrypts AES key using RSA (User A's public key)
- User A decrypts:
 - ✓ AES key using their private RSA key
 - ✓ Message using the decrypted AES key

Let's implement below steps to reach the goal.

1) generate_keys.py - User A generates RSA key pair

```
File Edit View Bookmarks Settings Help
[parrot@dell]~/media/parrot/storage/Final
$python3 generate_keys.py
[+] RSA key pair generated.
[parrot@dell]~/media/parrot/storage/Final
$ls
generate_keys.py private_key.pem public_key.pem README.txt
[parrot@dell]~/media/parrot/storage/Final
$cat generate_keys.py
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

# Generate private key
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

# Save private key
with open("private_key.pem", "wb") as f:
    f.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.TraditionalOpenSSL,
        encryption_algorithm=serialization.NoEncryption()
    ))

# Save public key
with open("public_key.pem", "wb") as f:
    f.write(public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ))

print("[+] RSA key pair generated.")
[parrot@dell]~/media/parrot/storage/Final
$
```

2) User B encrypts a message using - encrypt_message.py

```
Final : bash — Konsole
File Edit View Bookmarks Settings Help
└─$ cat encrypt_message.py
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.asymmetric import padding as rsa_padding
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives import padding as sym_padding
from os import urandom
import sys

# Load message
with open("message.txt", "rb") as f:
    message = f.read()
# Generate AES key and IV
aes_key = urandom(32) # AES-256
iv = urandom(16)
# Pad and encrypt message
padder = sym_padding.PKCS7(128).padder()
padded_message = padder.update(message) + padder.finalize()
cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv))
encryptor = cipher.encryptor()
ciphertext = encryptor.update(padded_message) + encryptor.finalize()
# Save encrypted message
with open("encrypted_message.bin", "wb") as f:
    f.write(iv + ciphertext)
# Load RSA public key
with open("public_key.pem", "rb") as f:
    public_key = serialization.load_pem_public_key(f.read())
# Encrypt AES key with RSA
encrypted_aes_key = public_key.encrypt(
    aes_key,
    rsa_padding.OAEP(
        mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
# Save encrypted AES key
with open("aes_key_encrypted.bin", "wb") as f:
    f.write(encrypted_aes_key)
print("[+] Message encrypted and AES key secured with RSA.")
[parrot@dell]~/media/parrot/storage/Final
└─$ python3 encrypt_message.py
[+] Message encrypted and AES key secured with RSA.
[parrot@dell]~/media/parrot/storage/Final
└─$ ls
aes_key_encrypted.bin  encrypted_message.bin  encrypt_message.py  generate_keys.py  message.txt  private_key.pem  public_key.pem  README.txt
[parrot@dell]~/media/parrot/storage/Final
└─$
```

3) User A decrypts AES key and message

```
Final : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@dell]~/media/parrot/storage/Final
$ cat decrypt_message.py
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.asymmetric import padding as rsa_padding
from cryptography.hazmat.primitives import serialization # Corrected import
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives import padding as sym_padding
# Load private RSA key
with open("private_key.pem", "rb") as f:
    private_key = serialization.load_pem_private_key(f.read(), password=None)
# Load encrypted AES key
with open("aes_key_encrypted.bin", "rb") as f:
    encrypted_aes_key = f.read()
# Decrypt AES key
aes_key = private_key.decrypt(
    encrypted_aes_key,
    rsa_padding.OAEP(
        mgf=rsa_padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
# Load encrypted message
with open("encrypted_message.bin", "rb") as f:
    data = f.read()
iv = data[:16] # First 16 bytes = IV
ciphertext = data[16:] # Rest = Encrypted message
# Decrypt message
cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv))
decryptor = cipher.decryptor()
padded_message = decryptor.update(ciphertext) + decryptor.finalize()
# Unpad the message
unpadder = sym_padding.PKCS7(128).unpadder()
message = unpadder.update(padded_message) + unpadder.finalize()
# Save the plaintext
with open("decrypted_message.txt", "wb") as f:
    f.write(message)
print("[+] Message decrypted successfully.")
[parrot@dell]~/media/parrot/storage/Final
$ python3 decrypt_message.py
[+] Message decrypted successfully.
[parrot@dell]~/media/parrot/storage/Final
$ ls
aes_key_encrypted.bin  decrypted_message.txt  decrypt_message.py  encrypted_message.bin  encrypt_message.py  generate_keys.py  message.txt
[parrot@dell]~/media/parrot/storage/Final
$
```

4) Let's check original and decrypted messages:

```
File Edit View Bookmarks Settings Help
[parrot@dell]-[/media/parrot/storage/Final]
└─ $ls -l
total 23
-rwxrwxrwx 1 root root 256 ɔ36 30 21:07 aes_key_encrypted.bin
-rwxrwxrwx 1 root root 92 ɔ36 30 21:13 decrypted_message.txt
-rwxrwxrwx 1 root root 1425 ɔ36 30 21:11 decrypt_message.py
-rwxrwxrwx 1 root root 112 ɔ36 30 21:07 encrypted_message.bin
-rwxrwxrwx 1 root root 1407 ɔ36 30 21:06 encrypt_message.py
-rwxrwxrwx 1 root root 777 ɔ36 30 20:51 generate_keys.py
-rwxrwxrwx 1 root root 92 ɔ36 30 21:00 message.txt
-rwxrwxrwx 1 root root 1675 ɔ36 30 20:51 private_key.pem
-rwxrwxrwx 1 root root 451 ɔ36 30 20:51 public_key.pem
-rwxrwxrwx 1 root root 2386 ɔ36 30 20:46 README.txt
[parrot@dell]-[/media/parrot/storage/Final]
└─ $cat message.txt
Top Secret Message !!!

Press the button and launch the nuclear bomb missile right now !!!
[parrot@dell]-[/media/parrot/storage/Final]
└─ $cat decrypted_message.txt
Top Secret Message !!!

Press the button and launch the nuclear bomb missile right now !!!
[parrot@dell]-[/media/parrot/storage/Final]
└─ $
```


Task 2: Secure File Exchange Using RSA + AES (8 pts)

Scenario: Alice wants to send Bob a secret file securely.

- Generate an RSA key pair for Bob (using OpenSSL or Python)
- Alice creates a plaintext message in a file called `alice_message.txt`
- Alice generates a random AES-256 key and IV
- Encrypt the file (`alice_message.txt`) using AES-256 with the generated key and IV
- Encrypt the AES key using Bob's RSA public key
- Bob decrypts the AES key using his RSA private key
- Bob uses the decrypted AES key and IV to decrypt the file and recover the original message
- After decryption, compute the SHA-256 hash of the file and compare it to the original hash for integrity verification

```
File Edit View Bookmarks Settings Help
Final - bash - Konsole
[parrot@dell]~/media/parrot/storage/Final
$python3 generate_keys.py
[+] Bob's RSA key pair generated and saved as public.pem and private.pem
[parrot@dell]~/media/parrot/storage/Final
$ls
alice_encrypt.py  alice_message.txt  bob_decrypt.py  generate_keys.py  private.pem  public.pem  README.md
[parrot@dell]~/media/parrot/storage/Final
$python3 alice_encrypt.py
Original SHA-256 hash: c511cba4007b2a8546541930a3bb6911fa61e4d2183fd4f8e8ad619108bb0954
[+] Message encrypted and AES key secured with Bob's public key
[parrot@dell]~/media/parrot/storage/Final
$ls
aes_key_encrypted.bin  alice_encrypt.py  alice_message.txt  bob_decrypt.py  encrypted_file.bin  generate_keys.py  private.pem  public.pem  README.md
[parrot@dell]~/media/parrot/storage/Final
$python3 bob_decrypt.py
Original hash: c511cba4007b2a8546541930a3bb6911fa61e4d2183fd4f8e8ad619108bb0954
Decrypted hash: c511cba4007b2a8546541930a3bb6911fa61e4d2183fd4f8e8ad619108bb0954
[+] Message decrypted successfully and integrity verified!
[parrot@dell]~/media/parrot/storage/Final
$cat alice_message.txt
Let's meet in the evening at the same time at the same location as usually.
I have Top Secret materials for you !!!
[parrot@dell]~/media/parrot/storage/Final
$cat decrypted_message.txt
Let's meet in the evening at the same time at the same location as usually.
I have Top Secret materials for you !!!
[parrot@dell]~/media/parrot/storage/Final
$
```

Task 3: TLS Communication Inspection & Analysis

Instructions:

1. Connect to any HTTPS website using openssl s_client
2. Extract and document:

Certificate chain (Root → Intermediate → Leaf)

```
File Edit View Bookmarks Settings Help
Final: openssl - Konsole
[parrot@de11]-[/media/parrot/storage/Final]
$openssl s_client -connect naec.ge:443
CONNECTED(00000003)
depth=2 C = US, ST = New Jersey, L = Jersey City, O = The USERTRUST Network, CN = USERTrust RSA Certification Authority
verify return:1
depth=1 C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo RSA Domain Validation Secure Server CA
verify return:1
depth=0 CN = *.naec.ge
verify return:1
---
Certificate chain
0 s:CN = *.naec.ge
  i:C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo RSA Domain Validation Secure Server CA
1 s:C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo RSA Domain Validation Secure Server CA
  i:C = US, ST = New Jersey, L = Jersey City, O = The USERTRUST Network, CN = USERTrust RSA Certification Authority
2 s:C = US, ST = New Jersey, L = Jersey City, O = The USERTRUST Network, CN = USERTrust RSA Certification Authority
  i:C = GB, ST = Greater Manchester, L = Salford, O = Comodo CA Limited, CN = AAA Certificate Services
3 s:C = GB, ST = Greater Manchester, L = Salford, O = Comodo CA Limited, CN = AAA Certificate Services
  i:C = GB, ST = Greater Manchester, L = Salford, O = Comodo CA Limited, CN = AAA Certificate Services
---
```

Cipher suite used

ECDHE-RSA-AES128-GCM-SHA256

TLS version

TLSv1.2

```
---
New, TLSv1.2, Cipher is ECDHE-RSA-AES128-GCM-SHA256
Server public key is 2048 bit
Secure Renegotiation IS supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol      : TLSv1.2
    Cipher       : ECDHE-RSA-AES128-GCM-SHA256
```

Client Hello

Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

its.handshake
[Icons]

No.	Time	* Source	Destination	Protocol	Length	Info
24	1.105854078	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
25	1.114580376	185.115.138.238	192.168.10.100	TLSv1.2	1434	Server Hello
33	1.117963819	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
35	1.124170974	192.168.10.100	185.115.138.238	TLSv1.2	192	Client Key Exchange, Change Cipher Spec, Encrypted Hand...
36	1.129084890	185.115.138.238	192.168.10.100	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Message
93	2.456418446	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
98	2.457962272	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
101	2.459058577	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
106	2.460191930	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
115	2.461722532	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
129	2.465439094	185.115.138.238	192.168.10.100	TLSv1.2	1434	Server Hello
143	2.465719680	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
147	2.467154950	192.168.10.100	185.115.138.238	TLSv1.2	192	Client Key Exchange, Change Cipher Spec, Encrypted Han...
151	2.470648430	185.115.138.238	192.168.10.100	TLSv1.2	1434	Server Hello
157	2.470709028	185.115.138.238	192.168.10.100	TLSv1.2	1434	Server Hello
165	2.471070216	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
167	2.471070271	185.115.138.238	192.168.10.100	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Message
171	2.471203456	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
173	2.472271430	192.168.10.100	185.115.138.238	TLSv1.2	192	Client Key Exchange, Change Cipher Spec, Encrypted Han...
177	2.472648657	185.115.138.238	192.168.10.100	TLSv1.2	1434	Server Hello
183	2.473388491	192.168.10.100	185.115.138.238	TLSv1.2	192	Client Key Exchange, Change Cipher Spec, Encrypted Han...

Frame 93: 721 bytes on wire (5768 bits), 721 bytes captured (5768 bits) on interface wlan0, id 0

Ethernet II, Src: IntelCor_97:ca:34 (34:cf:f6:97:ca:34), Dst: Shenzhen_19:21:7c (40:a5:ef:19:21:7c)

Internet Protocol Version 4, Src: 192.168.10.100, Dst: 185.115.138.238

Transmission Control Protocol, Src Port: 43498, Dst Port: 443, Seq: 1, Ack: 1, Len: 655

Transport Layer Security

- ↳ TLSv1.2 Record Layer: Handshake Protocol: Client Hello
 - Content Type: Handshake (22)
 - Version: TLS 1.0 (0x0301)
 - Length: 650
- ↳ Handshake Protocol: Client Hello
 - Handshake Type: Client Hello (1)
 - Length: 646
 - Version: TLS 1.2 (0x0303)

0000 40 a5 ef 19 21 7c 34 cf f6 97 ca 34 08 00 45 00 0...14...4...E

Handshake Protocol: Label
Packets: 4517 - Displayed: 31 (0.7%) - Dropped: 0 (0.0%)
Profile: Defa...

Server Certificate

```
➤ $openssl s_client -connect naec.ge:443
CONNECTED(00000003)
depth=2 C = US, ST = New Jersey, L = Jersey City, O = The USERTRUST Network, CN = USERTrust RSA Certification Authority
verify return:1
depth=1 C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo RSA Domain Validation Secure Server CA
verify return:1
depth=0 CN = *.naec.ge
verify return:1
---
Certificate chain
 0 s:CN = *.naec.ge
  i:C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo RSA Domain Validation Secure Server CA
 1 s:C = GB, ST = Greater Manchester, L = Salford, O = Sectigo Limited, CN = Sectigo RSA Domain Validation Secure Server CA
  i:C = US, ST = New Jersey, L = Jersey City, O = The USERTRUST Network, CN = USERTrust RSA Certification Authority
 2 s:C = US, ST = New Jersey, L = Jersey City, O = The USERTRUST Network, CN = USERTrust RSA Certification Authority
  i:C = GB, ST = Greater Manchester, L = Salford, O = Comodo CA Limited, CN = AAA Certificate Services
 3 s:C = GB, ST = Greater Manchester, L = Salford, O = Comodo CA Limited, CN = AAA Certificate Services
  i:C = GB, ST = Greater Manchester, L = Salford, O = Comodo CA Limited, CN = AAA Certificate Services
---
Server certificate
-----BEGIN CERTIFICATE-----
MITIGzCCBQuGAWIBAgIQI6DpbBMtUoT8VlhDANpgATANBgqhkiG9w0BAQsFADCB
jzELMAKGA1UEBhMCR0IxGzAZBgNVBAgTEkdyZWZ0ZXIqTWFuY2hlc3RlcjEQAQA4G
A1UEBxMHU2FzZm9yZDEYMBYGA1UEChMPU2VjdgLnbyBMaw1pdGVkHtCwNQYDVQDD
Ey5TZWN0aWdvIFJlTQSBEB21haW4gVmFsaWRhdGlvbiBTZWNIcmUgU2VydMvYIENB
MB4XDTI0MDUxMzAwMDAwMFOxDTI1MDUxMzIzNTk1OVowFDESMBAGA1UEAwWJk15u
YVWjLmdlMIIBIjANBgqhkiG9w0BAQEFAAQCAQ8AMIIBCGKCAQEAz40VS0PcR7D
Li+q5XvFlq2FeHpBxCG05R+3SEoL9BFXLbR1wJLJvCKSa50YKlw9PM8FaeC0gpAl
Om0WzuU12LrKRISsNLk4pvE1KfkhGMATlJApn9SKN4ozYrxc98pr1uovb/AwnJj+
HP2Rrdx29mKfWf02UPKfhhf6BojWfxllb16+rQ03rwBPjSHE8f0ZlQdpQfMdkJpJG
L1bZTVRqEsLGqMFLTGZy5zsELZDNWld+sw5jVW3euLSodAWHXReAJhAc79gQjCjM
C0+S+YPo/eS0+Lh2bwMrcccc+l0xEbUNl9J9A//wEzfEC14deEbvwbS79VLEgCLMR
lBFHFcPQaQIDAQABO4IC8zCCAU8WwYDVR0jBBGwFoAUjYxexFStiuf36Zv5mwXh
uAGNYeWwHQYDVR00BBYEFId2VMA0A/Wsvpl875A8XQoz5dqnMA4GA1UdDwEB/wQE
AwIFoDAMBGMNVHRMBAf8EAjAAMB0GA1UdJQQwMB0GCCsGAQUFBwMBBggrBgEFBQcD
AjBjBgNVHSAEQjBAMDDQCysGAQQBsJEBAgIHMCCUwIwYIKwYBBQUHAgEWF2h0dHBz
018vc2VjdGlnby5jb20vQ1BTMAgGBmeBDAECAATCBhAYIKwYBBQUHAQEEDB2ME8G
CCsGAQUFBzAChkNodHRw018vY3J0LnNlY3RpZ28uY29tL1NlY3RpZ29SU0FEb21h
aW5WYXxpZGF0aW9uU2VjdgXJlU2VydMvYQ0EuY3J0MCMGCCsGAQUFBzABhhhdodHRw
018vb2NzcC5zZWN0aWdvLmNvYyBAdBgNVHREEFjAUggkqLn5hZWwMuZ2WCB25hZWwMu
Z2UwggF9BgorBgEEAdZ5AgQCBIIBbQSCAwKBZwB2AM8RVu7VLnyv84db2Wkum+ka
cWdKsBfsrAH5W3f0zDsIAAABj3E/W/AAAAQDAEcwRQIgwKjovg+Tnu14f07B1fYSa
cR3oXgeTccQH9ek8uV9uexYCIQCN60QK2nhFR0x3f38DEwSASxAgReLNqeE3gbG6
6CIMdQB1AKLjCuRF772tm3447Udnd1PXgluElNcrXhssxLlQpEfnAAABj3E/W7MA
AAQDAEYwRAIgYRpY2QPrF55cJyGrIA4tA23fShB8kYJCgMX33401uAsCIFW3snGB
oN8aWcpeY0o6RhQ0xBQgCYBTZ8o/tfFL53wsAHYATnWjJ1yaEMM4W2zU3z9S6x3w
4I4bjWnAsfKsWKA0d8AAAGPcT9bIAAABAMARzBFAIAIue0Sfrq1RX0oU23qAQDZ
o6jLe09EWta3T/ABqm7zPAIhaJuLsq56FoPzX0pd1bvphClhXyVx1cPXSE9n0dVL
mm2LMA0GCSqGSIb3DQEBGwUAA4IBAQB8n6MjfxH0x1dkgEgUVxaHbCLBRHN8L3r
IU0x0LYt0unzeohlbaQL2q2SmVroYwF2XGohcLEqR2KQRARvNXpUdzvB9yJn+Ll
rS+x5aZa7vtEL0IR50LN3lt813utJ4H9a94D3lQdSHxbYe8Y1x+nn6XY7vCG+bYW
2ndys7X3F5sxejuWSS0vfpRXe3S08uXsLFAPj55IW4BxPT0PMONJ3RV5n/HSWiY2
E47Q43s1dRipgCQA7IRk7qM0i7HfRWyDUQtKCKXN+EJf8NKB7QY+Vhz0LEmrH+B3
XCKp//cHG/g/uu16qDKrerFU6HTR3hlt0k4Z9tE3ZSKCrqgwdrL
-----END CERTIFICATE-----
subject=CN = *.naec.ge
```


File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Its handshake.certificate

No.	Time	Source	Destination	Protocol	Length	Info
33	1.117963819	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
143	2.465719680	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
165	2.471070216	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
171	2.471203456	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
194	2.476679538	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
222	2.484249829	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done

Handshake Protocol: Certificate
Handshake Type: Certificate (11)
Length: 5640
Certificates Length: 5637
Certificates (5637 bytes)
Certificate Length: 1575
Certificate: 308206233082050ba003020102021023a0e96c132d5284fc56584300da6001300d06092a... (id-at-commonName=*.naec.ge)
signedCertificate
version: v3 (2)
serialNumber: 0x23a0e96c132d5284fc56584300da6001
signature (sha256WithRSAEncryption)
Algorithm Id: 1.2.840.113549.1.1.11 (sha256WithRSAEncryption)
issuer: rdnSequence (0)
rdnSequence: 5 items (id-at-commonName=Sectigo RSA Domain Validation Secure Server CA,id-at-organizationName=Sectigo Limited,id-at-local...
RDNSequence item: 1 item (id-at-countryName=GB)
RDNSequence item: 1 item (id-at-stateOrProvinceName=Greater Manchester)
RDNSequence item: 1 item (id-at-localityName=Salford)
RDNSequence item: 1 item (id-at-organizationName=Sectigo Limited)
RDNSequence item: 1 item (id-at-commonName=Sectigo RSA Domain Validation Secure Server CA)
validity
notBefore: utcTime (0)

Frame (686 bytes) Reassembled TCP (5649 bytes)
Content Type (tls.record.content_type), 1 byte(s)

Packets: 4517 - Displayed: 6 (0.1%) - Dropped: 0 (0.0%) Profile: Default

Key Exchange

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

*wlan0

Its handshake

No.	Time	Source	Destination	Protocol	Length	Info
24	1.105854078	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
25	1.114580376	185.115.138.238	192.168.10.100	TLSv1.2	1434	Server Hello
33	1.117963819	185.115.138.238	192.168.10.100	TLSv1.2	686	Certificate, Server Key Exchange, Server Hello Done
35	1.124170974	192.168.10.100	185.115.138.238	TLSv1.2	192	Client Key Exchange, Change Cipher Spec, Encrypted Hand...
36	1.129084890	185.115.138.238	192.168.10.100	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Message
93	2.456418446	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello
98	2.457962272	192.168.10.100	185.115.138.238	TLSv1.2	721	Client Hello

Frame 33: 686 bytes on wire (5488 bits), 686 bytes captured (5488 bits) on interface wlan0, id 0
Ethernet II, Src: Shenzhen_19:21:7c (40:a5:ef:19:21:7c), Dst: IntelCor_97:ca:34 (34:cf:f6:97:ca:34)
Internet Protocol Version 4, Src: 185.115.138.238, Dst: 192.168.10.100
Transmission Control Protocol, Src Port: 443, Dst Port: 43496, Seq: 5473, Ack: 656, Len: 620
[5 Reassembled TCP Segments (5649 bytes): #25(1272), #27(1368), #29(1368), #31(1368), #33(273)]
Transport Layer Security
TLSv1.2 Record Layer: Handshake Protocol: Certificate
Transport Layer Security
TLSv1.2 Record Layer: Handshake Protocol: Server Key Exchange
Content Type: Handshake (22)
Version: TLS 1.2 (0x0303)
Length: 333
Handshake Protocol: Server Key Exchange
Handshake Type: Server Key Exchange (12)
Length: 329
EC Diffie-Hellman Server Params
Curve Type: named_curve (0x03)
Named Curve: secp256r1 (0x0017)
Pubkey Length: 65
Pubkey: 043421f322792a631a0868a4de1802aa9b1777b52b1093238d2f425f69bf455960e0579...
Signature Algorithm: rsa_pkcs1_sha256 (0x0401)
Signature Hash Algorithm Hash: SHA256 (4)
Signature Hash Algorithm Signature: RSA (1)
Signature Length: 256
Signature: 774987e6d0d4d737f89e19b22c7680dda07cb8344e180b3195e23340268ee1f0736afb2...
TLSv1.2 Record Layer: Handshake Protocol: Server Hello Done

Frame (686 bytes) Reassembled TCP (5649 bytes)
Record Layer (tls.record), 338 byte(s)

Packets: 4517 - Displayed: 31 (0.7%) - Dropped: 0 (0.0%) Profile: Default

A TLS handshake is required to establish encryption parameters and provide authentication when a client and server start a secure communication session, like one via HTTPS. When the client sends the server a "Client Hello" message, the handshake begins. A list of acceptable cipher suites (encryption algorithms), the client's randomly generated nonce (referred to as the "client random"), the supported TLS version, and several TLS extensions, including Server Name Indication (SNI), which informs the server which domain the client is attempting to reach, are all included in this message. The server replies to the Client Hello by sending a series of handshake messages, one of which is the "Server Certificate." Usually certified by a reputable Certificate Authority (CA), this certificate is a digital document that includes the server's public key and identity information, including its domain name. The client verifies this certificate by making sure it matches the domain the client wanted to connect to, hasn't expired, and was issued by a reliable CA. This phase guards against man-in-the-middle attacks and guarantees the server's legitimacy. The client moves on to the key exchange stage when the certificate has been validated. Elliptic Curve Diffie-Hellman Ephemeral (ECDHE), which allows both sides to agree on a shared secret over an unsecured channel, is typically used for this in contemporary TLS versions. To make sure that only the authentic server can decode it with its private key, the client encrypts its part of the key exchange using the server's public key. All subsequent messages are symmetrically encrypted using the shared session key that is calculated by the client and server using their individual inputs. Through this procedure, the ensuing communication is guaranteed to be encrypted, authenticated, and shielded from manipulation or eavesdropping.

Briefly describe how TLS provides confidentiality and integrity

By integrating encryption, authentication, and message verification methods, TLS (Transport Layer Security) guarantees the confidentiality and integrity of data sent across a network. Symmetric encryption techniques, like AES, are used to achieve confidentiality once a secure handshake has created a shared secret session key between the client and server. All application data is encrypted using this key, which is only known by both parties. This makes sure that the data cannot be read even if a third party intercepts the transmission. The exchange is secured by the server's public key, which is transmitted within a digital certificate. During the key exchange phase, the symmetric key itself is safely negotiated, usually using Diffie-Hellman or Elliptic Curve Diffie-Hellman (ECDHE). Integrity is ensured by employing Authenticated Encryption with Associated Data (AEAD) ciphers such as AES-GCM or by incorporating a Message Authentication Code (MAC). Each message produced by these methods contains a cryptographic checksum, which the recipient uses to confirm that the data was not changed in transit. The message is rejected and the MAC validation fails if even one bit is altered. In order to verify the identity of the communication partners, TLS additionally allows digital signatures and certificates for server (and optionally client) authentication. This stops man-in-the-middle attacks and impersonation. To further avoid replay attacks, TLS uses nonces and preserves sequence numbers, making it impossible for an attacker to reuse outdated or duplicated communications. When combined, these features enable TLS to provide a strong degree of security by preventing unauthorized parties from reading or altering data while it is in transit.

Deliverables:

- Screenshots of openssl output and Wireshark capture
- Summary document: `tls_summary.txt`

Task 4: Email Encryption and Signature Simulation (PGP or S/MIME)

1. Generate key pairs using GPG or OpenSSL for PGP/S/MIME
2. Sign and encrypt a text file as "Alice"
3. Verify and decrypt it as "Bob"

```
Final : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@dell]~/media/parrot/storage/Final
$ ./generate_keys.sh
Generating Alice's GPG key...
gpg: key 611064538E88BE51 marked as ultimately trusted
gpg: revocation certificate stored as '/home/parrot/.gnupg/openpgp-revocs.d/E4A889C5A0DEE4ABE382CEBB611064538E88BE51.rev'
Generating Bob's GPG key...
gpg: key 2624059F814AC62F marked as ultimately trusted
gpg: revocation certificate stored as '/home/parrot/.gnupg/openpgp-revocs.d/83492A10424AB17FDC8C13A82624059F814AC62F.rev'
Key generation complete. Public and private keys exported.
[parrot@dell]~/media/parrot/storage/Final
$ ./alice_encrypt.sh
gpg: key 2624059F814AC62F: "Bob <bob@example.com>" not changed
gpg: Total number processed: 1
gpg:      unchanged: 1
usage: gpg [options] --sign --encrypt [filename]
Message signed and encrypted by Alice as signed_message.asc
[parrot@dell]~/media/parrot/storage/Final
$ ./alice_encrypt.sh
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid: 4  signed: 0  trust: 0-, 0q, 0n, 0m, 0f, 4u
✓ Message signed and encrypted successfully as 'signed_message.asc'
[parrot@dell]~/media/parrot/storage/Final
$ ./bob_decrypt.sh
gpg: key 2624059F814AC62F: "Bob <bob@example.com>" not changed
gpg: key 2624059F814AC62F: secret key imported
gpg: Total number processed: 1
gpg:      unchanged: 1
gpg:      secret keys read: 1
gpg:      secret keys unchanged: 1
gpg: key 611064538E88BE51: "Alice <alice@example.com>" not changed
gpg: Total number processed: 1
gpg:      unchanged: 1
gpg: encrypted with 2048-bit RSA key, ID 2624059F814AC62F, created 2025-04-30
      "Bob <bob@example.com>"
gpg: Signature made 2025 0406 30 030000, 23:33:47 +04
gpg:      using RSA key E4A889C5A0DEE4ABE382CEBB611064538E88BE51
gpg:      issuer "alice@example.com"
gpg: Good signature from "Alice <alice@example.com>" [ultimate]
Message decrypted and signature verified. Output saved to decrypted_message.txt
[parrot@dell]~/media/parrot/storage/Final
$ ls
alice_encrypt.sh  alice_public.asc  bob_private.key  decrypted_message.txt  original_message.txt
alice_private.key  bob_decrypt.sh    bob_public.asc   generate_keys.sh       signed_message.asc
```

```
Final : bash — Konsole
File Edit View Bookmarks Settings Help
[parrot@dell]~/media/parrot/storage/Final
$ ls
alice_encrypt.sh  alice_public.asc  bob_private.key  decrypted_message.txt  original_message.txt
alice_private.key  bob_decrypt.sh    bob_public.asc   generate_keys.sh       signed_message.asc
[parrot@dell]~/media/parrot/storage/Final
$ cat original_message.txt
This is a secret message from Alice to Bob.
[parrot@dell]~/media/parrot/storage/Final
$ cat decrypted_message.txt
This is a secret message from Alice to Bob.
```


Digital signatures play a crucial role in verifying the authenticity and integrity of a message by proving that it was indeed created by the claimed sender. In the context of public key cryptography, a digital signature is generated using the sender's private key and verified using their public key. When a sender like Alice wants to sign a message, she uses her private key to produce a unique signature derived from the content of the message. This signature is mathematically tied to both the message and her private key. When Bob, the recipient, receives the signed message, he uses Alice's public key to verify the signature. If the verification succeeds, it assures Bob that the message was indeed signed by someone in possession of Alice's private key—presumably Alice herself—and that the message has not been altered in transit. Bob will be reassured that the message was signed by someone who actually had Alice's private key—likely Alice herself—and that it wasn't tampered with during transit if the verification is successful. This procedure guarantees integrity because any alteration to the message would make the signature invalid and authentication because only Alice should have access to her private key. Bob can use a web of trust or certificate authority, depending on the technology (such as PGP or S/MIME), to confirm Alice's public key if he doesn't already trust it. This trust paradigm strengthens faith in the message's origin. For secure communications, digital contracts, and electronic authentication procedures, digital signatures provide a robust cryptographic way to verify the sender's identity and ensure that the received message has not been altered.

Task 5: Hashing & Integrity Check Utility

1. Create a Python script that:
 - Takes a file and computes SHA-256, SHA-1, and MD5 hashes
 - Stores hashes in a .json file
2. Simulate tampering by modifying the file and recomputing hashes
3. Script should detect the change and flag a warning

```
Final : bash — Konsole
File Edit View Bookmarks Settings Help
[✗]—[parrot@dell]—[/media/parrot/storage/Final]
→ $python hash_util.py save original.txt
[+] Hashes saved to hashes.json
[parrot@dell]—[/media/parrot/storage/Final]
→ $python hash_util.py check tampered.txt
[!] MD5 mismatch!
    Expected: ac49e13c2f618384deef951138a45be2
    Found:    c7361dcedc8738606f1aca30ef0bb559
[!] SHA1 mismatch!
    Expected: 7fbc4bf8ca122738e8c6f07075e32617ebf2c325
    Found:    cf6f0aaa81145e6551eaf84a7ffa2857f1de13f4
[!] SHA256 mismatch!
    Expected: ba0d24f2282a977d33031bb900c59a14646cf261fe5a4cd568b92e77cbb5f09f
    Found:    d4d7e5b512a9e75d7814ac8194645b76ec7874d31a1c905d0400fd15344bd456
[✗] Integrity check failed.
[parrot@dell]—[/media/parrot/storage/Final]
→ $cat original.txt
Agent 007, Be Careful !!!
[parrot@dell]—[/media/parrot/storage/Final]
→ $cat tampered.txt
Agent 007, You Don't Worry.
[parrot@dell]—[/media/parrot/storage/Final]
→ $python hash_util.py check original.txt
[✓] Integrity check passed.
[parrot@dell]—[/media/parrot/storage/Final]
→ $
```